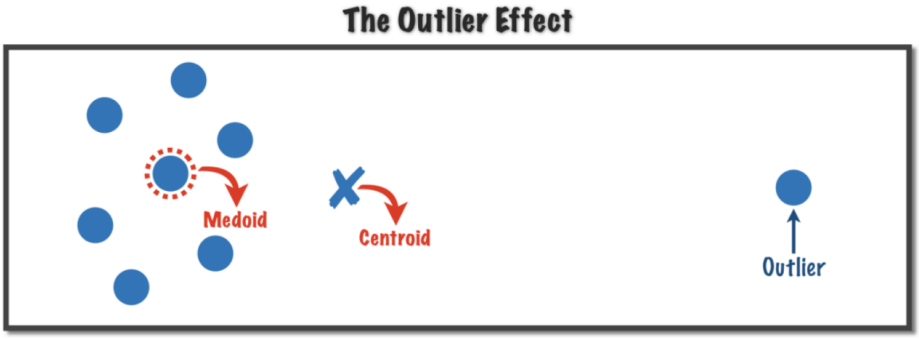
KMeans衍生

KMedoids

KMeans算法对异常值很敏感，因为每个簇的中心点计算是簇中所有观察值求平均，数据集中的极端数值会很大程度上干扰聚类效果。KMedoids是一个方法去客服这个问题。



中位数对极端值没有那么敏感，我们需要做的只是调整我们的更新步骤，从求平均切换为求中位数。比如我们实现KMeans的时候允许用户传入一个超参数，然后算法内部选择每次更新使用什么方法求新的中心点。

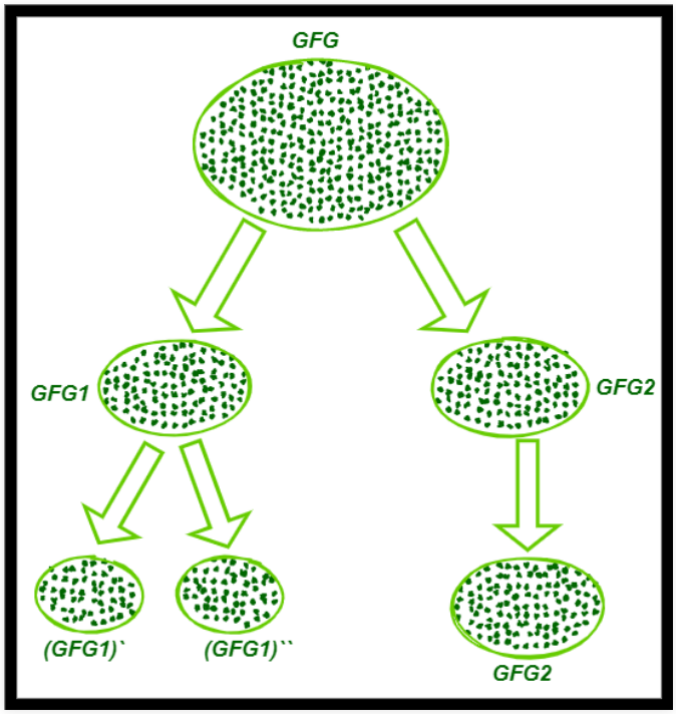
二分KMeans

为克服K-Means算法收敛于局部最小值问题，提出了二分K-Means算法。

二分K-Means算法首先将所有点作为一个簇，然后将该簇使用KMeans算法（K=2）一分为二。之后选择其中一个簇继续进行二分，选择哪一个簇进行二分取决于哪个簇有更大得SSE值。上述基于SSE的划分过程不断重复，直到得到用户指定的簇数目为止。

当然这里还有一种更简单的选择哪一个簇进行二分的策略，就是看在目前所有簇中哪个簇包含的样本点更多，就直接去二分哪个簇，这个应该比计算每个簇的SSE值然后比较要来的更快，而且大多数情况下会得到相似的结果。

SXT 尚学堂 bjsxt.com 百战程序员 让人人享有高品质教育



KMeans++

这个算法思想也很简单，与原始Kmeans唯一不同的是选择初始点的方式。而聚类效果不好，往往就是初始中心点分布不够均匀导致的。

为了克服上面提到的缺点，我们使用K-means++，这个算法通过一个更聪明的初始化中心点方法确保改进聚类的质量。除了初始化，算法其它和标准的K-means算法一样。所以K-means++就是标准的K-means算法加上了一个更好的初始化中心点的方法。

初始化中心点的步骤

1. 从数据集中随机选择第一个中心点
2. 对于每个数据点计算到它最近邻的已知中心点的距离
3. 从数据集中选择下一个中心点，是一个概率化选择，被选择的概率和上一步计算出来的距离成正比(例如，某个点到最近邻的已知中心点距离最远，它有最大的可能性被选为下一个中心点)
4. 重复第二步和第三步直到K个中心点被采样

TradeOff

虽然 K-means++ 算法初始化计算更费时与标准 K-means 算法相比，但是优化到收敛的运行时间大大减小了对于 K-means++ 来说。这是因为初始中心点的选择看起来已经是在不同的簇里面了。

elkan KMeans

在传统的K-Means算法中，我们在每轮迭代时，要计算所有的样本点到所有的质心的距离，这样会比较的耗时。那么，对于距离的计算有没有能够简化的地方呢？elkan K-Means算法就是从这块入手加以改进。它的目标是减少不必要的距离的计算。那么哪些距离不需要计算呢？

elkan K-Means利用了两边之和大于等于第三边，三角形不等式，来减少距离的计算。

第一种规律：我们知道，两边之和大于第三边

假设x是一个点，b和c是中心点；如果$d(b,c) \geq 2d(x,b)$，那么$d(x,c) \geq d(x,b)$

$$\begin{aligned} d(x,c) + d(x,b) &\geq d(b,c) \\ d(x,c) &\geq d(b,c) - d(x,b) \\ d(x,c) &\geq 2d(x,b) - d(x,b) \quad \text{if } d(b,c) \geq 2d(x,b) \\ d(x,c) &\geq d(x,b) \end{aligned}$$

第二种规律：依旧，两边之和大于第三边

假设x是一个点，b和c是中心点；那么$d(x,c) \geq max\{0, d(x,b) - d(b,c)\}$

$$\begin{aligned} d(x,c) + d(b,c) &\geq d(x,b) \\ d(x,c) &\geq d(x,b) - d(b,c) \\ \text{Also,} \quad d(x,c) &\geq 0 \end{aligned}$$

> 以上推导来自下面这篇论文第2章{Applying the triangle inequality}

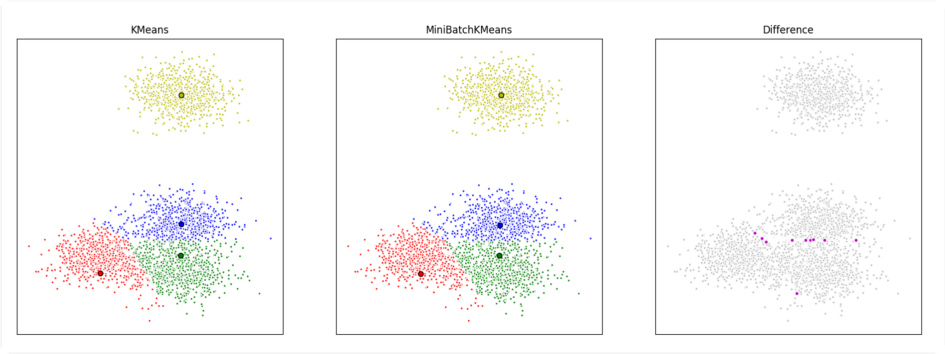
> https://cseweb.ucsd.edu//~elkan/kmeansicml03.pdf
>
> 如果对具体elkan算法流程感兴趣，请查看论文中的第3章节{The new algorithm}

利用上边的两个规律，elkan K-Means比起传统的K-Means迭代速度有很大的提高。但是如果我们的样本的特征是稀疏的，有缺失值的话，这个方法就不使用了，此时某些距离无法计算，则不能使用该算法。

Mini Batch KMeans

传统的K-Means算法中需要计算所有样本点到所有质心的距离，计算复杂度较高。如果样本量非常大的情况下，比如数据量达到10万，特征在100以上，此时用传统K-Means算法非常耗时。因此有了一种**分批处理**的改进算法Mini Batch K-Means。

Mini Batch K-Means算法是K-Means算法的变种，Mini Batch K-Means比K-Means有更快的收敛速度，但同时也降低了聚类的效果，但是在实际项目中却表现得不明显，有差异的基本都是聚类边界上的点。这是一张k-means和mini batch k-means的实际效果对比图。



该算法的迭代步骤有两步：

1. 从数据集中随机抽取一些数据形成小批量，把他们分配给最近的质心
2. 更新质心

与K均值算法相比，数据的更新是在每一个小的样本集上。对于每一个小批量，通过计算平均值得到更新质心，并把小批量里的数据分配给该质心，随着迭代次数的增加，这些质心的变化是逐渐减小的，直到质心稳定或者达到指定的迭代次数，停止计算。

KMeans小结

K-Means是个简单实用的聚类算法，这里对K-Means的优缺点做一个总结。

K-Means的主要优点：

1. 原理比较简单，实现也是很容易，收敛速度快（在大规模数据集上收敛较慢，可尝试使用Mini Batch K-Means算法）。
2. 聚类效果较优。
3. 算法的可解释度比较强。
4. 主要需要调参的参数仅仅是簇数k。

K-Means的主要缺点：

1. K值的选取不好把握（实际中K值的选定是非常难以估计的，很多时候，事先并不知道给定的数据集应该分成多少个类别才最合适。）
2. 对于不是凸的数据集比较难收敛（这时密度的聚类算法更加适合，比如DBSCAN算法，后面再介绍DBSCAN算法）
3. 如果各隐含类别的数据不平衡，比如各隐含类别的数据量严重失衡，或者各隐含类别的方差不同，则聚类效果不佳。
4. 采用迭代方法，得到的结果只是局部最优。（可以尝试采用二分K-Means算法）
5. 对噪音和异常点比较敏感。（改进1：离群点检测的LOF算法，通过去除离群点后再聚类，可以减少离群点和孤立点对于聚类效果的影响；改进2：改成求点的中位数，这种聚类方式即K-Mediods聚类）
6. 对初始值敏感（初始聚类中心的选择，可以尝试采用二分K-Means算法或K-Means++算法）
7. 时间复杂度高O(nkt)，其中n是对象总数，k是簇数，t是迭代次数。
8. 只适用于数值型数据，只能发现球型类簇。

实时效果反馈

1. 关于KMeans++，以下说法不正确的是:

A KMeans++是改进了KMeans算法初始中心点的方式

B KMeans++通过选择离已知中心点最远的样本点作为下一中心点

C KMeans++通过概率化选择初始中心点

D KMeans++是一个一个中心点依次选取出来的

2. 关于KMeans衍生的各种聚类算法，以下说法不正确的是:

A 二分KMeans是在KMeans聚类算法后再把一些簇进行二分

B KMedoids主要针对解决噪音和异常点比较敏感问题

C elkan KMeans提高的是划分样本时计算距离的效率

D Mini Batch KMeans是应用分批处理的思想提高计算效率

答案

1=>B　　2=>A